Homework 2 — Solutions

CS-UY 2413: Design & Analysis of Algorithms

Fall 2026 • Prof. Lisa Hellerstein

Recall: $f(n) = o(g(n))$ means that for *every* constant $c > 0$ there exists an n_0 such that $0 \le f(n) < c\,g(n)$ for all $n \ge n_0$.

1. Simplify each of the following expressions.

   Throughout this problem we use the identity

   $$n^{\log_b a} \;=\; a^{\log_b n},$$

   which holds for all positive a, b, n with $b \neq 1$. It is worth memorising: it lets us move an awkward exponent onto a friendlier base. We also use the special case $\log_b b = 1$, and the power rule $\log_b\big(x^k\big) = k \log_b x$.

   (a) $15^{\log_{15} 235}$

   Solution. Apply the identity with $n = 15$, $b = 15$ and $a = 235$:

   $$15^{\log_{15} 235} \;=\; 235^{\log_{15} 15} \;=\; 235^1 \;=\; 235.$$

   Answer: 235.

   (b) $\log_2 64^{100}$

   Solution. Bring the exponent down with the power rule, then write 64 as a power of 2:

   $$\log_2 64^{100} \;=\; 100 \log_2 64 \;=\; 100 \log_2 2^6 \;=\; 100 \cdot 6 \;=\; 600.$$

   Answer: 600.

(c) $49^{\log_7 10}$

Solution. Apply the identity with $n = 49$, $b = 7$ and $a = 10$, then write 49 as a power of 7:

$$49^{\log_7 10} \;=\; 10^{\log_7 49} \;=\; 10^{\log_7 7^2} \;=\; 10^2 \;=\; 100.$$

Answer: 100.

2. For each of the following, indicate whether $f = o(g)$ (little-oh), $f = \omega(g)$ (little-omega), or $f = \theta(g)$. No justification is necessary. For example, if $f(n) = n^3$ and $g(n) = n$, you would answer $f(n) = \omega(g(n))$.

   Although no justification is required, we include a short one for each part. Throughout we use the limit test: if $\lim_{n\to\infty} f(n)/g(n)$ is 0, then $f = o(g)$; if it is ∞, then $f = \omega(g)$; and if it is a positive number, then $f = \theta(g)$.

   (a) $f(n) = 26n^2 + 34n + 8, \quad g(n) = 2n^3$

   Solution. The dominant term of f is $26n^2$, which grows more slowly than n^3:

$$\lim_{n\to\infty} \frac{26n^2 + 34n + 8}{2n^3} \;=\; \lim_{n\to\infty} \left(\frac{13}{n} + \frac{17}{n^2} + \frac{4}{n^3} \right) \;=\; 0.$$

   Answer: $f(n) = o(g(n))$.

   (b) $f(n) = n^{24}, \quad g(n) = 3^n$

   Solution. Any polynomial grows more slowly than any exponential with base greater than 1, no matter how large the degree of the polynomial is. So

$$\lim_{n\to\infty} \frac{n^{24}}{3^n} \;=\; 0.$$

   (One way to see this is to apply L'Hôpital's rule 24 times. The numerator becomes the constant 24!, while the denominator stays $(\ln 3)^{24}\, 3^n$.)

   Answer: $f(n) = o(g(n))$.

   (c) $f(n) = n \log n, \quad g(n) = n^2$

   Solution. Cancel a factor of n. Since $\log n$ grows more slowly than n, we get

$$\lim_{n\to\infty} \frac{n \log n}{n^2} \;=\; \lim_{n\to\infty} \frac{\log n}{n} \;=\; 0.$$

   Answer: $f(n) = o(g(n))$.

(d) $f(n) = n^{\log_2 n}, \quad g(n) = 2^{(\log_2 n)^2}$

Solution. Write the base n as a power of 2, using $n = 2^{\log_2 n}$:

$$n^{\log_2 n} \;=\; \left(2^{\log_2 n}\right)^{\log_2 n} \;=\; 2^{(\log_2 n)(\log_2 n)} \;=\; 2^{(\log_2 n)^2}.$$

So $f(n) = g(n)$ exactly, and $f(n)/g(n) = 1$ for all n.

Answer: $f(n) = \theta(g(n))$.

(e) $f(n) = \sqrt{n}, \quad g(n) = \log n$

Solution. Any positive power of n, even a small one like $n^{1/2}$, eventually dominates $\log n$. By L'Hôpital's rule (taking log to be ln; a different base only changes a constant factor),

$$\lim_{n\to\infty} \frac{\sqrt{n}}{\ln n} \;=\; \lim_{n\to\infty} \frac{\frac{1}{2}n^{-1/2}}{1/n} \;=\; \lim_{n\to\infty} \frac{\sqrt{n}}{2} \;=\; \infty.$$

Answer: $f(n) = \omega(g(n))$.

3. Let

$$f(n) = 9\log_8 n \qquad \text{and} \qquad g(n) = \log_2 n.$$

Show that there is a constant c such that

$$(f(n))^2 = c(g(n))^2$$

for all $n > 0$, and find the value of c.

Solution. Using the change-of-base formula,

$$\log_8 n = \frac{\log_2 n}{\log_2 8} = \frac{\log_2 n}{3}.$$

Therefore,

$$\begin{aligned} f(n) &= 9\log_8 n \\ &= 9\left(\frac{\log_2 n}{3}\right) \\ &= 3\log_2 n \\ &= 3g(n). \end{aligned}$$

Squaring both sides gives

$$(f(n))^2 = (3g(n))^2 = 9(g(n))^2.$$

Thus the desired constant is

$$\boxed{c = 9}.$$

4. Let $g_1(n), g_2(n), \ldots, g_k(n)$ each satisfy $g_i(n) = o(n^2)$, and define

$$g(n) \;=\; \sum_{i=1}^{k} g_i(n).$$

(a) Show that $g(n) = o(n^2)$ when $k = 3$.

Solution. With $k = 3$ we have

$$g(n) \;=\; \sum_{i=1}^{3} g_i(n) \;=\; g_1(n) + g_2(n) + g_3(n).$$

Let $c > 0$ be arbitrary. Since $g_i(n) = o(n^2)$, the definition of little-oh applies to *every* positive constant, and in particular to the constant $c/3$. So for each $i \in \{1, 2, 3\}$ there exists an $n_i \in \mathbb{N}$ such that

$$g_i(n) \;<\; \frac{c}{3}\, n^2 \qquad \text{for all } n \geq n_i.$$

Note for later that this inequality is *strict*.

These three bounds hold simultaneously once n is at least as large as all three thresholds, so let

$$n_0 \;=\; \max\{n_1,\, n_2,\, n_3\}.$$

Then for every $n \geq n_0$ we may add the three inequalities:

$$\begin{aligned} g(n) \;=\; g_1(n) + g_2(n) + g_3(n) \;&<\; \frac{c}{3}n^2 + \frac{c}{3}n^2 + \frac{c}{3}n^2 \\ &=\; \left(\frac{c}{3} + \frac{c}{3} + \frac{c}{3}\right) n^2 \\ &=\; c\, n^2. \end{aligned}$$

We have therefore shown that for an arbitrary $c > 0$ there exists an $n_0 \in \mathbb{N}$ with $g(n) < c\, n^2$ for all $n \geq n_0$. Since c was arbitrary, this is precisely the definition of little-oh, and so

$$g(n) \;=\; o\big(n^2\big)\,. \qquad \blacksquare$$

Remark. The same argument works for any fixed constant k: split the budget as c/k per term and take $n_0 = \max\{n_1, \ldots, n_k\}$. What makes it work is that k is a constant, so the maximum is over a fixed finite collection of thresholds. Part (b) shows what goes wrong once that is no longer true.

(b) Does the same conclusion hold when $k = n$? That is, if $g_i(n) = o(n^2)$ for every i, must $g(n) = \sum_{i=1}^{n} g_i(n)$ also be $o(n^2)$?

Solution. No. The number of summands now grows with n, so the argument from part (a) breaks down, and in fact the statement is false.

Take the counterexample $g_i(n) = n$ for all i. Each such function is indeed $o(n^2)$, since

$$\lim_{n\to\infty} \frac{n}{n^2} \;=\; \lim_{n\to\infty} \frac{1}{n} \;=\; 0.$$

But summing n copies of n gives

$$g(n) \;=\; \sum_{i=1}^{n} n \;=\; n \cdot n \;=\; n^2.$$

And n^2 is *not* $o(n^2)$. To see this, recall that little-oh requires the inequality $g(n) < c\,n^2$ to hold for every constant $c > 0$. Taking $c = 1$ would require

$$n^2 \;<\; n^2,$$

which is false for every n. So no valid n_0 exists for $c = 1$, and the definition fails.

Answer: No — counterexample $g_i(n) = n$ for all i, which gives $g(n) = n^2$.

5. Consider the recurrence

$$T(n) \;=\; \begin{cases} 4\,T\!\left(\dfrac{n}{3}\right) + 2n, & n > 1, \\ 1, & n = 1, \end{cases}$$

and assume that n is a power of 3.

(a) Compute the value of $T(9)$, using the recurrence. Show your work.

Solution. We work upward from the base case. Since $9 = 3^2$, we first need $T(3)$, and for that we need $T(1)$:

$$\begin{aligned} T(1) \;&=\; 1, \\ T(3) \;&=\; 4\,T(1) + 2 \cdot 3 \;=\; 4 \cdot 1 + 6 \;=\; 10, \\ T(9) \;&=\; 4\,T(3) + 2 \cdot 9 \;=\; 4 \cdot 10 + 18 \;=\; 58. \end{aligned}$$

Answer: $T(9) = 58$.

(b) Specify the size of input, number of nodes, time per node, and total time for level i of the recursion tree (where level i is not a base case level).

Solution. The root is level 0. It has input size n and does $2n$ non-recursive work. Each node has 4 children, and each child's input is one third the size of its parent's. Going down one level therefore divides the input size by 3 and multiplies the number of nodes by 4. At level i, the input size is $n/3^i$ and there are 4^i nodes. Each node does $2 \cdot (n/3^i)$ non-recursive work, so the level as a whole does

$$4^i \cdot \frac{2n}{3^i} \;=\; 2n \left(\frac{4}{3}\right)^i .$$

Level	Size of input	Number of nodes	Time per node	Total time for level
0	n	1	$2n$	$2n$
1	$n/3$	4	$2n/3$	$2n \cdot (4/3)$
2	$n/9$	16	$2n/9$	$2n \cdot (4/3)^2$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
i	$n/3^i$	4^i	$2n/3^i$	$2n\,(4/3)^i$

Answer: At level i: size of input $n/3^i$; number of nodes 4^i; time per node $2n/3^i$; total time for the level $2n\,(4/3)^i$.

(c) Use the recursion tree approach to obtain a closed-form expression for $T(n)$, and check it at $n = 9$.

Solution. *Height of the tree.* The input size at level i is $n/3^i$. It reaches the base case size 1 when $3^i = n$, that is, when $i = \log_3 n$. So levels $0, 1, \ldots, \log_3 n - 1$ are non-base levels, and level $\log_3 n$ is the base case level.

Base case level. Level $\log_3 n$ has $4^{\log_3 n}$ nodes, each costing $T(1) = 1$. Using the identity $n^{\log_b a} = a^{\log_b n}$ from Problem 1,

$$4^{\log_3 n} \;=\; n^{\log_3 4},$$

so the base case level contributes $n^{\log_3 4}$ in total.

Non-base levels. Summing the last column of the table over levels 0 through $\log_3 n - 1$ gives a geometric series with ratio $4/3$. Recall that $\sum_{i=0}^{m-1} r^i =$

$(r^m - 1)/(r-1)$ for $r \neq 1$. So

$$\sum_{i=0}^{\log_3 n - 1} 2n \left(\frac{4}{3}\right)^i = 2n \cdot \frac{(4/3)^{\log_3 n} - 1}{4/3 - 1}$$
$$= 6n \left(\left(\frac{4}{3}\right)^{\log_3 n} - 1 \right).$$

We simplify the power. Since $3^{\log_3 n} = n$ and $4^{\log_3 n} = n^{\log_3 4}$,

$$\left(\frac{4}{3}\right)^{\log_3 n} = \frac{4^{\log_3 n}}{3^{\log_3 n}} = \frac{n^{\log_3 4}}{n}.$$

Substituting this in, the non-base levels contribute

$$6n \left(\frac{n^{\log_3 4}}{n} - 1 \right) = 6\, n^{\log_3 4} - 6n.$$

Total. Adding the base case level to the non-base levels,

$$T(n) = \left(6\, n^{\log_3 4} - 6n\right) + n^{\log_3 4} = 7\, n^{\log_3 4} - 6n.$$

Check at $n = 9$. By the same identity, $9^{\log_3 4} = 4^{\log_3 9} = 4^2 = 16$. So

$$T(9) = 7 \cdot 16 - 6 \cdot 9 = 112 - 54 = 58,$$

which agrees with part (a).

Answer: $T(n) = 7\, n^{\log_3 4} - 6n$.

6. For each recurrence, determine whether Our Master Theorem applies. If it does, give the solution in Θ notation.

(a)
$$T(n) = 3T(n/3) + n^{1/3}.$$

Solution. Here
$$a = 3, \qquad b = 3, \qquad d = \frac{1}{3}.$$
Also,
$$\log_b a = \log_3 3 = 1.$$
Since
$$d = \frac{1}{3} < 1 = \log_b a,$$
Our Master Theorem gives
$$\boxed{T(n) = \Theta(n)}.$$

(b)

$$T(n) = 8T(n/2) + 8n^4.$$

Solution. Since $8n^4 = \Theta(n^4)$, we have

$$a = 8, \qquad b = 2, \qquad d = 4.$$

Also,

$$\log_b a = \log_2 8 = 3.$$

Since

$$d = 4 > 3 = \log_b a,$$

Our Master Theorem gives

$$\boxed{T(n) = \Theta(n^4)}.$$

(c)

$$T(n) = T(n/2) + 2.$$

Solution. Since $2 = \Theta(1) = \Theta(n^0)$, we have

$$a = 1, \qquad b = 2, \qquad d = 0.$$

Also,

$$\log_b a = \log_2 1 = 0.$$

Thus

$$d = \log_b a = 0.$$

Therefore,

$$\boxed{T(n) = \Theta(\log n)}.$$

(d)

$$T(n) = nT(n/2) + n.$$

Solution. Our Master Theorem does not apply.

The master method applies to recurrences of the form $aT(n/b) + \theta(n^d)$ where a,b, and d are constants. However, here $a = n$, which is not a constant, so the recurrence is not of the required form.

(e)

$$T(n) = 4T(n^{1/3}) + n.$$

Solution. Our Master Theorem does not apply.

The recursive subproblem has size $n^{1/3}$ rather than n/b for some constant $b > 1$, so the recurrence is not of the required form.

(f)

$$T(n) = 2T(n/4) + \log n.$$

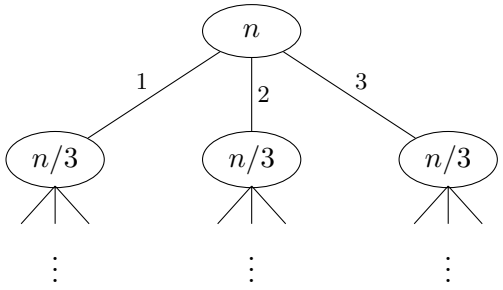
Solution. Our Master Theorem does not apply.

The non-recursive term $\log n$ is not $\Theta(n^d)$ for any constant $d \geq 0$.

7. Consider the variant of merge sort that splits the input into *three* parts rather than two, sorts each recursively, and merges the results.

   (a) Write down a recurrence for the running time $T(n)$.

   Solution. We split each array of size n into 3 subproblems, each of size $n/3$, and recurse on each:



   This accounts for the $3\,T(n/3)$ term. It remains to count the non-recursive work done at each node, that is, the cost of merging the three sorted pieces back together.

   From our study of merge sort we know that merging two sorted arrays, $\textsc{Merge}(A, B)$, takes $\Theta(n)$ time. Here we perform two such merges: first the left and middle pieces, and then that result with the right piece. Two merges, each costing $\Theta(n)$, still total up to $\Theta(n)$, Thus, the total non-recursive work per level is therefore $\Theta(n)$.

   Combining the recursive and non-recursive work we get:

$$T(n) \;=\; 3\,T\Big(\frac{n}{3}\Big) + \Theta(n).$$

Answer: $\mathrm{T(n)} \;=\; 3\,\mathrm{T}\big(\frac{n}{3}\big) + \Theta(n)$.

(b) Solve the recurrence.

Solution. We apply the Master Theorem, which states the following. If a recurrence has the form

$$T(n) \;=\; a\,T\Big(\frac{n}{b}\Big) + \Theta\Big(n^d\Big)\,,$$

then

$$T(n) \;=\; \begin{cases} \Theta\big(n^d\big)\,, & \text{if } a < b^d, \\ \Theta\big(n^d \log n\big)\,, & \text{if } a = b^d, \\ \Theta\big(n^{\log_b a}\big)\,, & \text{if } a > b^d. \end{cases}$$

Reading the parameters off the recurrence from part (a), we have $a = 3$, $b = 3$ and $d = 1$. Comparing a against b^d:

$$b^d \;=\; 3^1 \;=\; 3 \;=\; a,$$

so we are in the second case, $a = b^d$. Therefore

$$T(n) \;=\; \Theta\Big(n^d \log n\Big) \;=\; \Theta(n \log n).$$

Answer: $T(n) = \Theta(n \log n)$.

Remark. This matches the running time of ordinary two-way merge sort. Splitting into three parts changes the constant factor but not the asymptotic running time, which is what we should expect: the recursion tree is shallower by a factor of $\log_2 3$, but each level still does $\Theta(n)$ work.

8. Consider the recurrence

$$T(n) \;=\; 4\,T\Big(\Big\lfloor \frac{n}{3} \Big\rfloor\Big) + 9n^2 \quad \text{for } n > 2, \qquad T(1) = T(2) = 8.$$

Using proof by induction and the definition of big-Oh, prove that $T(n) = O(n^2)$. Do not use the Master Method.

Solution. By the definition of big-Oh, we must find constants $c > 0$ and n_0 such that

$$T(n) \;\le\; c\,n^2 \qquad \text{for all } n \ge n_0.$$

We take $n_0 = 1$. We can leave c unknown for now. As we go through the proof, we can write down every condition that c must satisfy and choose a value of c that satisfies all of them at the end.

Base cases ($n = 1$ and $n = 2$).

$$T(1) \;=\; 8 \;\le\; c \cdot 1^2 \;=\; c \qquad \text{holds provided } c \ge 8,$$

$$T(2) \;=\; 8 \;\le\; c \cdot 2^2 \;=\; 4c \qquad \text{holds provided } c \ge 2.$$

So the base cases hold as long as $c \ge 8$.

Induction hypothesis. Let $n \ge 3$, and assume that $T(k) \le c\,k^2$ for every integer k with $1 \le k < n$.

Induction step. We show that $T(n) \le c\,n^2$. Let $k = \lfloor n/3 \rfloor$. Since $n \ge 3$, we have $k \ge 1$. Also $k \le n/3 < n$. So k lies in the range covered by the induction hypothesis, which gives $T(k) \le c\,k^2$. Using this and the recurrence,

$$\begin{aligned} T(n) \;&=\; 4\,T\Big(\Big\lfloor \frac{n}{3} \Big\rfloor\Big) + 9n^2 \\ &\le\; 4c \Big\lfloor \frac{n}{3} \Big\rfloor^2 + 9n^2 \qquad \text{(by our induction hypothesis)} \\ &\le\; 4c \Big(\frac{n}{3}\Big)^2 + 9n^2 \qquad \text{(since } 0 \le \lfloor n/3 \rfloor \le n/3\text{)} \\ &=\; \frac{4c}{9}\,n^2 + 9n^2. \end{aligned}$$

We want this to be at most $c\,n^2$. Dividing through by $n^2 > 0$, it suffices that

$$\frac{4c}{9} + 9 \;\le\; c \quad\Longleftrightarrow\quad 9 \;\le\; \frac{5c}{9} \quad\Longleftrightarrow\quad c \;\ge\; \frac{81}{5} \;=\; 16.2.$$

So the induction step goes through as long as $c \ge 81/5$.

Choosing c. The base cases need $c \ge 8$, and the induction step needs $c \ge 16.2$. Taking

$$c \;=\; 17$$

satisfies both. (Any $c \ge 81/5$ would work.) With this choice, both base cases hold. The induction step shows that if $T(k) \le c\,k^2$ holds for every k with $1 \le k < n$, then $T(n) \le c\,n^2$ holds as well, for every $n \ge 3$.

Therefore, with $c = 17$ and $n_0 = 1$, we have $T(n) \le c\,n^2$ for all $n \ge n_0$, and so

$$T(n) \;=\; O\big(n^2\big)$$

9. Now consider the recurrence

$$T(n) \;=\; 4T\Big(\Big\lfloor \frac{n}{2} \Big\rfloor\Big) + n \quad \text{for } n > 1, \qquad T(1) = 1.$$

We know from the Master Theorem that $T(n) = O(n^2)$, but we will prove it using induction.

(a) Try to prove that

$$T(n) \leq cn^2$$

for all $n \geq n_0$. Show that the induction step fails regardless of the value of c.

Solution. Suppose our induction hypothesis is that

$$T(k) \leq ck^2$$

for every integer $k < n$. Since $\lfloor n/2 \rfloor < n$, we can apply the induction hypothesis to the recursive call:

$$\begin{aligned} T(n) &= 4T\Big(\Big\lfloor \frac{n}{2} \Big\rfloor\Big) + n \\ &\leq 4c \Big\lfloor \frac{n}{2} \Big\rfloor^2 + n \\ &\leq 4c \left(\frac{n}{2}\right)^2 + n \\ &= cn^2 + n. \end{aligned}$$

To complete the induction step, we would need

$$cn^2 + n \leq cn^2.$$

Subtracting cn^2 from both sides gives

$$n \leq 0,$$

which is false for every $n > 1$.

Notice that the value of c disappears completely from this condition. Therefore, choosing a larger value of c cannot fix the problem. The induction step fails for every choice of c.

Conclusion: The statement $T(n) \leq cn^2$ is true for a suitable c, but this particular induction hypothesis is not strong enough to prove it.

(b) Instead, prove the stronger statement

$$T(n) \leq cn^2 - n$$

for all $n \geq 1$, using a suitable value of c.

Solution. We take $n_0 = 1$ and will prove

$$T(n) \leq cn^2 - n$$

for every $n \geq 1$.

First, we determine a suitable value of c from the base case.

Since $n_0 = 1$, our base case is $n = 1$. We need

$$T(1) \leq c(1)^2 - 1.$$

Since $T(1) = 1$, this becomes

$$1 \leq c - 1,$$

so

$$c \geq 2.$$

Thus, $c = 2$ is the smallest convenient choice that makes the base case work. We will show that this same value also makes the induction step work.

Our claim is therefore

$$T(n) \leq 2n^2 - n \qquad \text{for all } n \geq 1.$$

Base case. When $n = 1$,

$$T(1) = 1 = 2(1)^2 - 1.$$

So the claim holds.

Induction hypothesis. Let $n > 1$, and assume that

$$T(k) \leq 2k^2 - k$$

for every integer k with $1 \leq k < n$.

Induction step. Let

$$k = \left\lfloor \frac{n}{2} \right\rfloor.$$

Since $n > 1$, we have $1 \leq k < n$, so the induction hypothesis applies:

$$T(k) \leq 2k^2 - k.$$

Because of the floor, we consider whether n is even or odd.

Case 1: n is even.

Let $n = 2m$. Then

$$\left\lfloor \frac{n}{2} \right\rfloor = m.$$

Using the recurrence and the induction hypothesis,

$$\begin{aligned} T(n) &= 4T(m) + n \\ &\leq 4(2m^2 - m) + 2m \\ &= 8m^2 - 2m. \end{aligned}$$

Since $n = 2m$,

$$2n^2 - n = 2(2m)^2 - 2m = 8m^2 - 2m.$$

Therefore,

$$T(n) \leq 2n^2 - n.$$

Case 2: n is odd.

Let $n = 2m + 1$. Then

$$\left\lfloor \frac{n}{2} \right\rfloor = m.$$

Again using the recurrence and the induction hypothesis,

$$\begin{aligned} T(n) &= 4T(m) + n \\ &\leq 4(2m^2 - m) + (2m + 1) \\ &= 8m^2 - 2m + 1. \end{aligned}$$

On the other hand,

$$\begin{aligned} 2n^2 - n &= 2(2m + 1)^2 - (2m + 1) \\ &= 8m^2 + 6m + 1. \end{aligned}$$

Since $m \geq 0$,

$$8m^2 - 2m + 1 \leq 8m^2 + 6m + 1.$$

Thus,

$$T(n) \leq 2n^2 - n.$$

Therefore, by induction,

$$T(n) \leq 2n^2 - n \qquad \text{for all } n \geq 1.$$

Finally, since

$$2n^2 - n \leq 2n^2,$$

we have

$$T(n) \leq 2n^2$$

for all $n \geq 1$. Hence

$$\boxed{T(n) = O(n^2)}.$$